

UNIVERSITI TUNKU ABDUL RAHMAN

ACADEMIC YEAR 2023/2024

MAY EXAMINATION

UCCD2103 OPERATING SYSTEMS

MONDAY, 13 MAY 2024

TIME : 2.00 PM – 4.00 PM (2 HOURS)

BACHELOR OF COMPUTER SCIENCE (HONOURS)
BACHELOR OF INFORMATION SYSTEMS (HONOURS)
COMPUTER ENGINEERING
BACHELOR OF INFORMATION SYSTEMS (HONOURS)
INFORMATION SYSTEMS ENGINEERING
BACHELOR OF INFORMATION TECHNOLOGY (HONOURS)
COMMUNICATIONS AND NETWORKING

Instructions to Candidates :

This question paper consists of THREE (3) questions in Section A and TWO (2) questions in Section B.

Answer **ALL** questions in **Section A** and **ONLY ONE (1)** question in **Section B**.
Each question carries 25 marks.

Should a candidate answer more than ONE (1) question in section B, marks will only be awarded for the FIRST ONE (1) question in that section in the order the candidate submits the answers.

Candidates are allowed to use calculator.

Answer questions only in the answer booklet provided.

UCCD2103 OPERATING SYSTEMS**Section A: Answer ALL questions.**

- Q1. (a) Draw and label the diagram of 7-State process model. (7 marks)
- (b) Referring to the following scenarios, list each state transition of Process A based on the 7-state model:
- (i) Process A is created and is dispatched to execute immediately. (1 mark)
 - (ii) Process A then waits for user input via the keyboard. (1 mark)
 - (iii) While waiting for that event to occur, another process, Process B, is dispatched to execute and immediately consume a large amount of memory. Process A is then suspended to virtual memory. (1 mark)
 - (iv) The user finishes inputting the required data into Process A. (1 mark)
 - (v) Process A is then selected to resume execution until finish. (1 mark)
- (c) Figure 1.1 shows the CPU and I/O utilizations of a uniprocessor computer system when Process C and D are executing. Process D has CPU and I/O instructions whilst Process C has only CPU instructions. By $t = 35\text{ms}$, both processes have completed their execution.

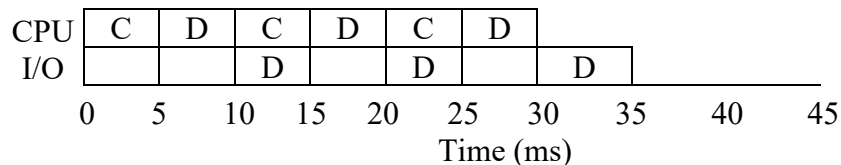


Figure 1.1: CPU and I/O utilizations of Process C and Process D.

Assume Process C and D are in READY state at $t = 0\text{ ms}$ and Process C is selected to execute first. Answer the following questions:

- (i) Name the system that matches the execution pattern shown in Figure 1.1. Briefly highlight the main advantage of that system. (3 marks)
- (ii) According to Figure 1.1, what are the turnaround times of the Process C and D, respectively? (4 marks)
- (iii) Draw the execution pattern of Process C and D if they were to execute on a uniprocessor simple batch system and then calculate their turnaround times. (6 marks)

[Total : 25 marks]

UCCD2103 OPERATING SYSTEMS

- Q2. (a) This question is about the Completely Fair Scheduler (CFS) in the Linux kernel. Assume that the system has a uniprocessor and the period for CFS is set at 20 milliseconds. There are three CPU-bound processes in Ready state: Processes E, F and G with the following parameters in Table 2.1.

Table 2.1: Parameters for Process E, F and G

Parameters	Process E	Process F	Process G
Virtual runtime (nanoseconds)	1,202,086	1,173,910	1,192,388
Nice value	+6	+1	-6

The minimum virtual runtime of the red-black tree is 1,173,910 nanoseconds and the CFS need to decide which process to execute. Using the guide provided in the Appendix A, answer the following questions:

- (i) In what sequence will Processes E, F and G be executed after the snapshot in Table 2.1? (3 marks)
 - (ii) What are the virtual runtimes of Processes E, F and G after one period? (6 marks)
 - (iii) The CFS favours tasks that use the least processing time in recent past. Explain why this scheduling algorithm favours I/O-bound programs and yet still could prevent CPU-bound programs into starvation. (4 marks)
- (b) Assume that your main memory has a total of 6 frames. Initially, frames #0 and #1 are assigned to Process A and frames #2 and #3 are assigned to Process B, respectively. Other frames have yet to be assigned to any process. Figure 2.1 shows the current state of the main memory where every frame has use-bit set to 0.

frame #0	
frame #1	
frame #2	
frame #3	
frame #4	
frame #5	

Figure 2.1: Current state of memory frames

Then, the following page trace occurs (process name and its respective page number):

A1, A3, B2, B8, B7, A0, A3, B7, B4, B7

Show the steps of each page replacement performed and the use-bit of each frame if the clock replacement policy is used with:

UCCD2103 OPERATING SYSTEMS**Q2. (b)(Continued)**

- (i) **Fixed** Allocation and **Local** Replacement Scope. (6 marks)
- (ii) **Variable** Allocation and **Local** Replacement Scope. Assume that the page replacement policy will immediately allocate an additional frame to a process that has two consecutive page faults, by selecting a free frame starting from frame #1 in increasing order. It will immediately reduce the allocation by one frame if a process has two consecutive page references, by freeing up a frame from that process where the page is the least recently used page. (6 marks)
- [Total : 25 marks]

- Q3. (a) Consider a disk head is now positioned at track 135 and has completed a previous request at track 120. The requests of track are in the following sequence:

75, 130, 175, 90, 125, 220, 200, 195

Given that the SCAN disk scheduling algorithm was implemented, draw the seek pattern on a graph starting from track 135. Let the x-axis and y-axis represents the sequence number and requested track number. (8 marks)

- (b) A copy of some sectors from disk will be stored in the disk cache temporarily and will be replaced over the time. One of the replacement policies uses the number of accesses to a block as the replacement criteria. The number of accesses of each block will be stored in their associated counter. A block with the smallest count will be replaced.
- (i) Briefly explain the problem that can occur in the above replacement policy. (3 marks)
- (ii) What is the replacement policy that can overcome the problem explained in part Q3(b)(i)? (1 mark)
- (c) File F has been stored into a disk with 24 blocks (Block 0 to Block 23) and the disk bit table, which currently only contains File F, is given as follow:

110000011010011110001110

Assume that *Indexed Allocation with Variable Length Portions* is implemented and Table 3.1 shows a portion of the file allocation table.

UCCD2103 OPERATING SYSTEMS**Q3. (c)(Continued)**

Table 3.1: File allocation table

File name	Index Block
File F	10
File G	X

- (i) List the contents of the Index Block for File F. (4 marks)
- (ii) A new file named File G is to be stored into the disk and 9 blocks are required to store File G. Given that the searching for free blocks to store File G will start from the first block, Block 0, and the first available free block will be used as the Index Block, what is the value of X in Table 3.1? (1 mark)
- (iii) List the contents of the Index Block for File G. (4 marks)
- (d) Consider a real-time monitoring system that continuously receive videos from various cameras with different resolution and length. All received videos will undergo initial processing and must be promptly stored on a disk as a backup. All the stored videos will not be retrieved for further processing and the videos will be deleted after 72 hours. Briefly discuss the suitability of contiguous allocation versus linked allocation for the system. (4 marks)

[Total : 25 marks]

UCCD2103 OPERATING SYSTEMS**Section B: Answer ANY ONE (1) question.**

- Q4. (a) Table 4.1 shows the current allocation of resources to each process, the estimated time remaining of each process and the resources that are still needed by each process to proceed and complete their execution. The resources currently available are $R1 = 2$, $R2 = 1$, $R3 = 0$ and $R4 = 0$. It is known that all the processes are deadlocked. Resolve the deadlock by *successively abort deadlocked processes with the most estimated time remaining*. Show the resources available after each abortion in your workings.

Table 4.1: Resources allocation of processes

Process	Estimated time remaining	Allocated				Needed			
		R1	R2	R3	R4	R1	R2	R3	R4
A	10	1	3	2	1	7	6	3	3
B	13	2	2	3	1	5	3	2	1
C	14	1	1	2	1	3	3	1	3
D	20	1	0	0	1	3	4	3	2
E	17	1	2	0	0	3	2	2	2

(9 marks)

- (b) Given the size of a user process is 16GB, page frame size is 4KB and each page table entry consumes 2 bytes, calculate the size of the root page table in two-level scheme. Show all your calculations. (4 marks)
- (c) A process can be divided into maximum 64 segments and Table 4.2 shows a portion of a segment table for a process. Translate the following virtual addresses to their corresponding physical addresses. Show all the relevant values in your answer.

Table 4.2: Portion of a segment table

Segment number	Base	Length (decimal)
25	0000 0010 0110 0100	1024
26	0000 0100 0010 0001	896
27	0000 0111 0011 0101	768

- (i) 0110011011101001 (3 marks)
- (ii) 0110111100000111 (3 marks)
- (d) (i) Briefly explain how the number of page frames allocated to a process relate to the thrashing problem. (3 marks)
- (ii) Briefly explain the reason for the existence of a *modify bit* in a page table entry. (3 marks)

[Total : 25 marks]

UCCD2103 OPERATING SYSTEMS

- Q5. (a) Given that a system has 1MB of memory available, apply the Buddy system allocation to the followings.

NOTE: Answer can also be shown in the form of binary tree.

- (i) Memory requests in the sequence shown in Table 5.1. (4 marks)

Table 5.1: Memory requests

Process	Request
A	50 KB
B	25 KB
C	200 KB
D	110 KB
E	130 KB
F	120 KB

- (ii) The memory partitions allocated to the process A, process B, process E and process F are released. (2 marks)

- (b) Explain race condition based on the pseudocode in Figure 5.1. Assume that both threads can complete all the computations within their time slices.

Global variable x = 2, y = 4, z;	
<pre>T1() { x = x + y / 2; z = x + y * x; }</pre>	<pre>T2() { x = x * y - 2; z = x * y - y; }</pre>

Figure 5.1: Pseudocode of two threads with global variables (5 marks)

- (c) Figure 5.2 shows the pseudocode of two processes. Considering any process can be selected to execute first and process switching can cause two processes become deadlocked, explain whether the processes in Figure 5.2 can become deadlocked. State the changes of the semaphores' value in your explanation. You may use the line number (L1, L2, etc.) in your answer.

semaphore s=1, t=2;		
	P()	Q()
L1	{	{
L2	semWait(t);	semWait(s);
L3	semWait(s);	semWait(t);
L4	//critical section	//critical section
L5	semSignal(s);	semSignal(t);
L6	semSignal(t);	semSignal(s);
L7	}	}

Figure 5.2: Pseudocode of process P and process Q

(7 marks)

UCCD2103 OPERATING SYSTEMS**Q5. (Continued)**

- (d) Figure 5.3 shows two threads, namely F1 and F2. Figure 5.4 shows the skeletons of the two threads. Thread F1 will read data from Buffer, perform the filter_1() operation on the data and place the output back into Buffer. Subsequently, thread F2 will read thread F1's output from Buffer and empty it. Thread F2 will perform the filter_2() operation and store the output into a file. After thread F2 completes its operation, thread F1 will be allowed to read new data from Buffer. Assume that Buffer is filled with data before both threads start and new data will be written into Buffer after thread F2 emptied the content of Buffer.

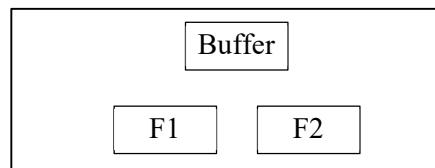


Figure 5.3: Two threads with a share buffer

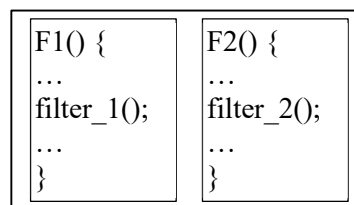


Figure 5.4: Skeletons of thread F1 and thread F2

- (i) What is the number of semaphores required to synchronise the execution of the two threads? (1 mark)
- (ii) Write the pseudocode for the two threads in Figure 5.4. Initialise each semaphore with the appropriate value. (6 marks)
- [Total : 25 marks]

UCCD2103 OPERATING SYSTEMS

APPENDIX A:

Linux Completely Fair Scheduler (CFS):

1. Tasks' time slice allocation:

$$f_i(t) = \frac{w_i}{\sum w_{rbtree}} \times p$$

$f_i(t)$ = time slice received by task i at time t ,
 w_i = weight of task i

$\sum w_{rbtree}$ = Total weight of tasks in red-black tree

p = period

2. Virtual runtime of task i in n -th iteration:

$$v_i(n) = v_i(n-1) + \left(\frac{\Delta e}{w_i} \right) \times w_{nice0}$$

$v_i(n-1)$ = virtual runtime of task i in previous iteration

Δe = elapsed time task i spent utilizing the processor during iteration n

w_i = weight of task i

w_{nice0} = weight assigned to nice 0 task = 1024

3. Mapping of nice value to weight:

```
static const int prio_to_weight[40] = {

    /* -20 */ 88761,    71755,    56483,    46273,    36291,
    /* -15 */ 29154,    23254,    18705,    14949,    11916,
    /* -10 */ 9548,     7620,     6100,     4904,     3906,
    /* -5  */ 3121,     2501,     1991,     1586,     1277,
    /* 0   */ 1024,     820,      655,     526,      423,
    /* 5   */ 335,      272,      215,     172,      137,
    /* 10  */ 110,      87,       70,      56,       45,
    /* 15  */ 36,       29,       23,      18,       15,
}
```